

**Universidad Nacional General Sarmiento**

**Sistemas Operativos y Redes II**

**Trabajo Práctico n°2: Seguridad**

**Autores/as:**

**Joaquin Muñoz**

**Francisco Javier Ruiz Lezcano**

**Gastón Sanchez**

**Mauricio Quevedo**

Buenos Aires - Argentina  
2026

|                                                                        |    |
|------------------------------------------------------------------------|----|
| <b>1. Introducción</b>                                                 | 3  |
| 1.1. Entorno                                                           | 3  |
| <b>2. Hardening</b>                                                    | 4  |
| 2.1. Objetivo                                                          | 4  |
| 2.2. Diseño del script                                                 | 4  |
| 2.3. Diseño del script                                                 | 5  |
| 2.3. Controles implementados                                           | 6  |
| 2.4. Ejecución y evidencias                                            | 7  |
| 2.5. Idempotencia                                                      | 9  |
| <b>3. AppArmor</b>                                                     | 9  |
| 3.3. Ejecución y evidencias                                            | 10 |
| 3.4. DAC frente a MAC                                                  | 12 |
| 3.5. Complain frente a enforce                                         | 12 |
| <b>4. Auditoría, syscalls e integridad</b>                             | 13 |
| 4.1. Reglas auditd                                                     | 13 |
| 4.2. Correlación entre strace y auditd                                 | 14 |
| 4.2.1 Análisis 5: correlación entre la syscall y el evento persistente | 16 |
| 4.3. Línea BASE y control de integridad con AIDE                       | 16 |
| 4.3.1 Objetivo                                                         | 16 |
| 4.3.2 Configuración de AIDE                                            | 17 |
| 4.3.3 Generación de la línea base                                      | 17 |
| 4.3.4 Cambios controlados                                              | 18 |
| 4.3.5 Limitación de AIDE                                               | 18 |
| <b>5. Escenario integrador</b>                                         | 19 |
| <b>6. Conclusión</b>                                                   | 21 |

# 1. Introducción

Este informe documenta el Trabajo Práctico 2 de Sistemas Operativos y Redes II. El trabajo aborda la seguridad de un sistema GNU/Linux desde tres perspectivas complementarias:

- **Hardening:** reducir configuraciones innecesariamente riesgosas.
- **Control de acceso obligatorio (AppArmor):** limitar lo que puede hacer una aplicación aunque los permisos tradicionales se lo permitan.
- **Observación (auditd, strace y AIDE):** conservar evidencia de lo que ocurre en el sistema.

El objetivo no es aplicar un benchmark completo. Para cada control explicamos qué riesgo reduce, cómo comprobamos su efecto y qué evidencia dejaría un incidente.

Las salidas completas del terminal no se reproducen en el texto. Están en la carpeta [evidencias/](#), organizada por parte, y en el informe citamos el archivo y el fragmento relevante. Cada evidencia se presenta indicando qué se intentó, qué resultado se esperaba, qué ocurrió y qué demuestra.

El trabajo se dividió por partes, y cada integrante generó las evidencias de la suya:

| Parte                      | Responsable | Evidencias                                        |
|----------------------------|-------------|---------------------------------------------------|
| 1. Hardening guiado        | Francisco   | <a href="#">evidencias/parte1_Fran/</a>           |
| 2. AppArmor                | Joaquín     | <a href="#">evidencias/parte2_Joaquin/</a>        |
| 3.1 y 3.2. auditd y strace | Mauricio    | <a href="#">evidencias/parte3.1_3.2_Mauricio/</a> |
| 3.3. AIDE                  | Gastón      | <a href="#">evidencias/parte3.3_Gaston/</a>       |

## 1.1. Entorno

Todo el trabajo se realizó en máquinas virtuales aisladas de VirtualBox y solo con los archivos ficticios de [/srv/tp2](#); no se usó ningún equipo personal ni datos reales. La VM tiene Ubuntu, 4 GB de RAM y un procesador virtual, con hostname [alumno-virtualbox](#) y usuario [alumno](#).

Además de la consola de la VM, habilitamos acceso por SSH desde el equipo anfitrión mediante reenvío de puertos de la red NAT ([127.0.0.1:2222](#) hacia el puerto 22 de la VM). Como pide la consigna, la consola se mantuvo disponible como acceso de respaldo antes de modificar la configuración de SSH.

El entorno se preparó con [scripts/preparar\\_entorno.sh --install-packages](#), que hace lo siguiente:

- instala `build-essential`, `apparmor-utils`, `auditd`, `aide`, `strace`, `libpam-pwquality` y `openssh-server`;
- compila e instala `tp2-reader` y `tp2-event` en `/usr/local/bin`;

- crea el usuario de laboratorio `tp2lab`, con la cuenta bloqueada para inicio de sesión por contraseña;
- prepara `/srv/tp2` con los archivos `publico.txt` y `confidencial.txt`, ambos `0644` y de `root:root`;
- habilita `auditd`;
- deja una fotografía del estado inicial en `/var/lib/tp2-lab/baseline.txt`.

Esa línea base (`evidencias/parte3.1_3.2_Mauricio/00_baseline.txt`) confirma que, antes de cualquier cambio del grupo, AppArmor estaba cargado (28 perfiles, 26 en modo enforce) y `auditd` estaba habilitado. Después verificamos el entorno con `tests/public_checks.sh --entorno` y creamos el snapshot `TP2_BASE`, que es el punto de restauración de todo el trabajo.

## 2. Hardening

### 2.1. Objetivo

La Parte 1 consistió en completar `hardening/hardening_base.sh` como `hardening/hardening.sh`. El script implementa cuatro controles que modifican el sistema y conserva un control de diagnóstico sobre las cuentas con UID 0. Acepta tres modos:

- `--check`: informa el estado de cada control.
- `--apply`: aplica solo lo que falta.
- `--restore`: devuelve el sistema al estado anterior al TP.

### 2.2. Diseño del script

Tres decisiones del marco provisto, que respetamos en los cuatro controles, explican el comportamiento que se ve en las evidencias.

**Fragmentos separados en lugar de editar archivos del sistema.** Cada control escribe un archivo propio llamado `99-tp2-hardening` dentro del directorio `.d/` que corresponde. Ningún archivo original de la distribución se modifica, y deshacer el cambio equivale a borrar o reponer un único archivo.

**Escritura atómica y respaldo único.** `write_file` escribe el contenido en un temporal dentro del mismo directorio, le asigna permisos y lo mueve sobre el destino con `mv`. Ningún servicio llega a leer un archivo a medio escribir.

Antes de escribir, `backup_file` registra el estado original del destino:

- si el archivo existía, guarda una copia y una marca `.existed`;
- si no existía, guarda una marca `.absent`.

Ese registro se hace una sola vez: si la marca ya existe, no se vuelve a tomar el respaldo. Así, una segunda ejecución de `--apply` nunca reemplaza el respaldo original por la versión ya endurecida.

**Decisión basada en el estado real.** Cada control decide qué hacer buscando con `grep -qx` la línea exacta que espera en su fragmento. No usa ningún registro de ejecuciones anteriores. Todas las acciones quedan en `/var/log/tp2-hardening.log` (permisos `0600`) con fecha, estado (`CHECK`, `APPLIED`, `SKIPPED`, `ERROR`) y control.

## 2.3. Diseño del script

Cada control es una función de Bash (`control_ssh`, `control_pwquality`, `control_umask`, `control_sysctl`) con la misma forma: define el archivo destino y su contenido, y después ramifica según el modo. El control de SSH es el más completo y sirve de ejemplo:

```
control_ssh() {
    local target="/etc/ssh/sshd_config.d/99-tp2-hardening.conf"
    local content='$PermitRootLogin no\n'
    real="$(path "$target")"
    if [[ "$MODE" == "--check" ]]; then
        [[ ! -f "$real" ]] && { mark_error "SSH" "Archivo $target no existe"; return; }
        grep -qx 'PermitRootLogin no' "$real" || { mark_error "SSH" "..."; return; }
        mark_checked "SSH" "Archivo $target existe y contiene PermitRootLogin no"
    elif [[ "$MODE" == "--apply" ]]; then
        if [[ -f "$real" ]] && grep -qx 'PermitRootLogin no' "$real"; then
            mark_skipped "SSH" "$target ya establece PermitRootLogin no"
            return
        fi
        backup_file "$target"
        write_file "$target" 0644 "$content"
        sshd -t || { mark_error "SSH" "sshd -t fallo despues de modificar $target"; return; }
        systemctl reload ssh
        mark_applied "SSH" "Creado/actualizado $target y ssh recargado"
    elif [[ "$MODE" == "--restore" ]]; then
        restore_file "$target"
        sshd -t || { mark_error "SSH" "sshd -t fallo despues de restaurar $target"; return; }
        systemctl reload ssh
        mark_applied "SSH" "Restaurado $target y ssh recargado"
    }
}
```

```
fi
}
```

El fragmento está condensado para el informe: en el script, cada condición ocupa un bloque `if` propio y los mensajes de error son completos.

En la rama `--apply`, el orden de las operaciones es lo que hace seguro al control:

1. Si el fragmento ya tiene la línea esperada, registra `SKIPPED` y termina sin tocar nada.
2. Si no, registra el estado original con `backup_file` y recién después escribe.
3. Valida la configuración completa con `sshd -t` antes de recargar el servicio.

## 2.3. Controles implementados

| Control                | Fragmento                                                                                                                                               | Riesgo que reduce                                                                                                                                                                                               | Cómo se valida                                                                                                                                                  |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UID 0                  | Ninguno (solo diagnóstico)                                                                                                                              | Una cuenta adicional con UID 0 tiene privilegios de root bajo otro nombre, y es una forma clásica de persistencia tras una intrusión.                                                                           | El script lista las cuentas con UID 0 distintas de root. No las corrige automáticamente, porque borrar una cuenta es una decisión que requiere análisis humano. |
| OpenSSH                | <code>/etc/ssh/sshd_config.d/99-tp2-hardening.conf</code> con <code>PermitRootLogin no</code>                                                           | Acceso remoto directo como root (ver Análisis 1).                                                                                                                                                               | <code>sshd -t</code> antes de recargar el servicio y <code>sshd -T</code> para ver la configuración efectiva.                                                   |
| Calidad de contraseñas | <code>/etc/security/pwquality.conf.d/99-tp2-hardening.conf</code> con <code>minlen = 12</code> , <code>minclass = 3</code> , <code>maxrepeat = 3</code> | Contraseñas cortas o predecibles, vulnerables a fuerza bruta o diccionario. Exige 12 caracteres, al menos 3 de las 4 clases (minúsculas, mayúsculas, dígitos, otros) y no más de 3 caracteres iguales seguidos. | Confirmar que <code>pam_pwquality.so</code> está activo en <code>/etc/pam.d/common-password</code> y que el fragmento contiene la política.                     |

| Control            | Fragmento                                                                                                                         | Riesgo que reduce                                                                                                                                                                                      | Cómo se valida                                                                              |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Umask              | <code>/etc/profile.d/99-tp2-hardening.sh</code> con <code>umask 027</code>                                                        | Archivos nuevos legibles por cualquier usuario del sistema. Con <code>027</code> , los archivos nuevos quedan <code>0640</code> y los directorios <code>0750</code> : sin ningún permiso para "otros". | Abrir una sesión de login nueva, ver la máscara y los permisos de un archivo recién creado. |
| Enlaces protegidos | <code>/etc/sysctl.d/99-tp2-hardening.conf</code> con <code>fs.protected_hardlinks=1</code> y <code>fs.protected_symlinks=1</code> | Ataques con enlaces en directorios compartidos como <code>/tmp</code> : un atacante crea un enlace con un nombre predecible para que un proceso privilegiado escriba en un archivo ajeno.              | Revisar el archivo en <code>sysctl.d</code> y los valores efectivos del kernel.             |

El control de SSH tiene una salvaguarda adicional. Después de escribir el fragmento ejecuta `sshd -t`, y solo recarga el servicio si la validación pasa. Si falla, registra **ERROR** y no recarga. Esto evita que un error de configuración deje la VM sin acceso remoto.

## 2.4. Ejecución y evidencias

Todas las ejecuciones se hicieron desde la raíz del repositorio, en el orden de la tabla. Los archivos están en `evidencias/parte1_Fran/`.

| Evidencia                       | Qué se intentó                             | Qué se esperaba                                | Qué ocurrió                                                                                                                                 | Qué demuestra                                                                                                                     |
|---------------------------------|--------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>02_hardening_check</code> | <code>--check</code> antes de aplicar nada | UID 0 correcto y los cuatro controles en error | <b>CHECK</b> en UID0 y <b>ERROR</b> en los demás porque no existe ningún fragmento.<br><code>Verificados=1</code><br><code>Errores=4</code> | El estado inicial no tenía ninguno de los controles. <code>--check</code> no cambió la configuración ( <code>Aplicados=0</code> ) |
| <code>03_hardening_apply</code> | Primer <code>--apply</code>                | Que se apliquen los cuatro controles           | Cuatro <b>APPLIED</b> , SSH recargado y valores de <code>sysctl</code> aplicados en caliente.                                               | El script crea solo lo que falta, y <code>sshd -t</code> aceptó la configuración antes de la recarga                              |

| Evidencia                 | Qué se intentó                                            | Qué se esperaba                                 | Qué ocurrió                                                                                                                                                   | Qué demuestra                                                                                                                              |
|---------------------------|-----------------------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                           |                                                           |                                                 | Aplicados=4<br>Errores=0                                                                                                                                      |                                                                                                                                            |
| 04_hardening_restore      | --restore                                                 | Volver al estado de la evidencia 02             | Cada fragmento se eliminó "porque no existía antes del TP". Aplicados=8<br>Errores=0                                                                          | --restore usa las marcas .absent: como en el estado original no había fragmentos, restaurar significa borrarlos                            |
| 05_hardening_idempotencia | Dos --apply seguidos                                      | Primero aplicar todo y después no repetir nada  | La primera corrida dio Aplicados=4. La segunda, 4 segundos después, dio cuatro SKIPPED y Omitidos=4                                                           | La primera corrida confirma que el restore había borrado los fragmentos (si no, habría dado SKIPPED). La segunda es la base del Análisis 2 |
| 06_hardening_validaciones | Leer los fragmentos y comprobar el efecto de cada control | Valores efectivos coherentes con los fragmentos | permitrootlogin no en la configuración efectiva, pam_pwquality.s o activo, máscara 0027 y archivo nuevo -rw-r-----.<br>--check dio Verificados=5<br>Errores=0 | Los controles no solo están escritos: están vigentes                                                                                       |

Dos aclaraciones sobre estas evidencias:

- **Aplicados=8 en el restore.** El resumen cuenta 8 y no 4 porque cada archivo restaurado registra dos líneas **APPLIED**: una de la función **restore\_file** y otra del control correspondiente. Son cuatro archivos contados dos veces.
- **El sysctl sin sudo de la evidencia 06.** Respondió "permiso denegado" porque en esta VM esas dos claves solo las puede leer root. Los valores efectivos los confirma el --check posterior, que corre como root y compara **sysctl -n** contra 1: "valores efectivos del kernel en 1".

Cómo --restore funcionó correctamente, no hizo falta recurrir al snapshot para revertir el hardening. Al final de la Parte 1 la VM quedó con los cuatro controles aplicados.

## 2.5. Idempotencia

Un script es idempotente si ejecutarlo varias veces deja el sistema en el mismo estado que ejecutarlo una vez. La evidencia 05 lo muestra con dos ejecuciones del mismo comando, separadas por 4 segundos:

```
22:44:08 [RESUMEN] [TP2] Aplicados=4 Omitidos=0 Verificados=1 Errores=0
```

```
22:44:12 [RESUMEN] [TP2] Aplicados=0 Omitidos=4 Verificados=1 Errores=0
```

La primera ejecución parte de un sistema sin fragmentos y los crea. La segunda encuentra los cuatro controles ya cumplidos y los registra como **SKIPPED**. No reescribe ningún archivo y no vuelve a recargar SSH.

El script no se comporta así porque "recuerde" que ya corrió. Antes de actuar, cada control comprueba el estado real del sistema: busca en su fragmento la línea exacta que espera. Si la línea está, no hay nada que hacer. Si se borrara un solo fragmento, el siguiente **--apply** recrearía únicamente ese y omitiría los otros tres.

La idempotencia también protege la reversión. Como **backup\_file** toma el respaldo solo la primera vez, ejecutar **--apply** muchas veces no reemplaza el estado original guardado por la versión endurecida. Gracias a eso, **--restore** sigue devolviendo el sistema al estado previo al TP, como mostró la evidencia 04.

## 3. AppArmor

Los archivos `publico.txt` y `confidencial.txt` tienen permisos `0644` y pertenecen a `root:root`. Con esos permisos, cualquier usuario del sistema puede leerlos. El objetivo de la Parte 2 fue escribir un perfil de AppArmor para `tp2-reader` que le permita leer el recurso público pero no el confidencial. Para demostrar que el bloqueo viene de AppArmor (MAC) y no de los permisos tradicionales (DAC), primero hubo que probar que DAC sí permitía la lectura.

También había que mostrar la diferencia entre los dos modos de un perfil:

- `complain`: registra lo que el perfil no permite, pero lo deja pasar.
- `enforce`: bloquea lo que el perfil no permite.

Completamos `apparmor/usr.local.bin.tp2-reader.base` y lo guardamos como `apparmor/usr.local.bin.tp2-reader`:

```
#include <tunables/global>

/usr/local/bin/tp2-reader flags=(attach_disconnected) {
    #include <abstractions/base>

    # Permitir la ejecucion y mapeo del propio binario
    /usr/local/bin/tp2-reader mr,

    # Permitir la lectura del archivo publico
    /srv/tp2/datos/publico.txt r,
}
```

| Regla                        | Para qué está                                                                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #include <abstractions/base> | Permisos mínimos que necesita cualquier programa en C: cargar la libc y el enlazador dinámico, leer <a href="#">/etc/ld.so.cache</a> , etc. Sin esto el programa no llega a arrancar. |
| /usr/local/bin/tp2-reader mr | <a href="#">r</a> para leer el binario y <a href="#">m</a> para mapearlo en memoria como ejecutable.                                                                                  |
| /srv/tp2/datos/publico.txt r | Única regla sobre los datos: lectura del archivo público.                                                                                                                             |

**No hay ninguna regla para `confidencial.txt`, a propósito.** AppArmor funciona como lista blanca: todo lo que el perfil no permite está denegado. Por eso no hace falta una regla [deny](#). La consigna además pedía no usarla, porque una regla [deny](#) explícita bloquea incluso en complain, y no se hubiera podido observar la diferencia entre los dos modos ante un permiso ausente.

### 3.3. Ejecución y evidencias

Seguimos la secuencia de la consigna:

1. Validar la sintaxis con [apparmor\\_parser -Q](#) antes de cargar.
2. Cargar el perfil con [apparmor\\_parser -r](#) y pasarlo a complain.
3. Leer los dos archivos y buscar el evento.
4. Pasar a enforce y repetir las lecturas.

El modo del perfil lo confirmamos en cada paso leyendo [/sys/kernel/security/apparmor/profiles](#). Los archivos están en [evidencias/parte2\\_Joaquin/](#).

Cada **.txt** tiene los comandos y sus salidas, y el **.png** con el mismo número es la captura de esa pantalla.

| Evidencia         | Qué se intentó                                                         | Qué se esperaba                                                       | Qué ocurrió                                                                                                                                                                                                                                                                                 | Qué demuestra                                                                                                   |
|-------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 01_sin_perfil_dac | Ver permisos y leer los dos archivos con el perfil descargado          | Que se lean los dos                                                   | Sin perfil cargado (el grep sobre perfiles no devolvió nada). Los dos archivos son <b>-rw-r--r-- root root</b> , el usuario es alumno (uid 1000) y <b>tp2-reader</b> leyó los dos                                                                                                           | DAC permite la lectura de <b>confidencial.txt</b> : alumno entra por la categoría "otros", que tiene <b>r--</b> |
| 02_complain       | Validar y cargar el perfil, pasarlo a complain y leer los dos archivos | Que se lean los dos, pero quede registrado el acceso no permitido     | <b>apparmor_parser -Q y -r</b> no devolvieron errores y el perfil quedó en ( <b>complain</b> ). Los dos archivos se leyeron. En <b>audit.log</b> quedó <b>apparmor="ALLOWED"</b> sobre <b>confidencial.txt</b> con <b>denied_mask="r"</b> (pid=3302)                                        | El kernel detecta que el acceso no está en el perfil y lo anota, pero no lo bloquea                             |
| 03_enforce        | Pasar a enforce y repetir las lecturas, también como root              | Que <b>publico.txt</b> se lea y <b>confidencial.txt</b> sea rechazado | El perfil quedó en ( <b>enforce</b> ). <b>publico.txt</b> se leyó. <b>confidencial.txt</b> dio "Permission denied" como alumno y también con sudo. En <b>audit.log</b> quedaron dos <b>apparmor="DENIED"</b> : pid=3318 con <b>fsuid=1000</b> (alumno) y pid=3320 con <b>fsuid=0</b> (root) | AppArmor bloquea el acceso que DAC permitía, y lo hace aunque el proceso corra como root                        |

Algunas aclaraciones sobre estas evidencias:

- **El evento no estaba en el journal del kernel.** La consigna propone buscarlo con **journalctl -k | grep apparmor**, pero ese comando no devolvió nada (evidencia 02). AppArmor informa sus decisiones a través del subsistema de auditoría del kernel. Cuando auditd está corriendo, el kernel le entrega esos registros a auditd y quedan en **/var/log/audit/audit.log**; solo van al journal del kernel si auditd no está activo. Como el preparador deja auditd habilitado, buscamos el evento en **audit.log**.

- **Líneas anteriores en los logs.** El grep sobre `audit.log` muestra las últimas coincidencias, y algunas son de una prueba previa del mismo día. Las de esta ejecución se identifican por el pid: 3302 en complain, y 3318 y 3320 en enforce.
- **Las líneas sueltas en la evidencia 01.** Debajo del primer comando aparecen "l.txt", "id" y otras. No son salida del grep: son el eco de los comandos, que se pegaron todos juntos mientras el primero todavía corría.

La regla que faltaba, y que habría permitido la lectura, es `/srv/tp2/datos/confidencial.txt r`. Al final de la Parte 2 el perfil quedó cargado en enforce, como lo requiere el escenario integrador.

### 3.4. DAC frente a MAC

Cuando `tp2-reader` abre un archivo, el kernel hace dos controles sobre la misma llamada `openat`:

1. **DAC:** compara el usuario del proceso con el dueño, el grupo y los bits de permiso del archivo. `confidencial.txt` es `0644` de `root:root`, y alumno no es el dueño ni está en el grupo root, así que cae en "otros", que tiene permiso de lectura. Este control pasa, y así lo muestra la evidencia 01: sin perfil, el contenido se imprime.
2. **MAC:** después, el kernel consulta a AppArmor a través de los hooks de LSM. AppArmor no mira al usuario sino al programa: busca en el perfil de `/usr/local/bin/tp2-reader` una regla que permita leer esa ruta. Para `publico.txt` la encuentra; para `confidencial.txt` no. Como el perfil está en enforce, la llamada vuelve con `EACCES`. `tp2-reader` lo muestra como "Permission denied" y termina con código 2.

La prueba como root lo confirma. Para DAC, root puede leer prácticamente cualquier archivo. Sin embargo, la evidencia 03 muestra el mismo rechazo con sudo, y el registro `DENIED` de ese intento tiene `fsuid=0`. El permiso de AppArmor depende del perfil del programa, no de quién lo ejecuta, y ningún usuario puede cambiarlo sobre sus propios archivos: solo el administrador, cargando o modificando el perfil.

### 3.5. Complain frente a enforce

Las evidencias 02 y 03 usan el mismo perfil, el mismo archivo y el mismo usuario. Lo único que cambia es el modo:

|                                          | complain (evidencia 02)            | enforce (evidencia 03)                 |
|------------------------------------------|------------------------------------|----------------------------------------|
| Lectura de <code>confidencial.txt</code> | Permitida: se imprime el contenido | Rechazada: Permission denied, código 2 |

|                              |                                        |                                         |
|------------------------------|----------------------------------------|-----------------------------------------|
| Registro en <b>audit.log</b> | apparmor="ALLOWED"                     | apparmor="DENIED"                       |
| Efecto para la aplicación    | Ninguno, funciona igual que sin perfil | Pierde el acceso que el perfil no le da |

En los dos casos el registro tiene **denied\_mask="r"**: el kernel identifica igual cuál es el permiso que falta. La diferencia es qué hace con esa información.

En la práctica, complain sirve para construir y probar un perfil sin romper la aplicación. Si el perfil se olvidara de algo que el programa necesita de verdad, en complain el programa seguiría funcionando y el registro **ALLOWED** avisaría qué regla agregar. En nuestro caso ese registro confirmó, antes de bloquear nada, que el único acceso fuera del perfil era la lectura de **confidencial.txt**. Pero complain no protege nada: el archivo confidencial se leyó igual. La protección real empieza recién al pasar a enforce, donde la misma ausencia de regla se convierte en un bloqueo.

## 4. Auditoría, syscalls e integridad

### 4.1. Reglas auditd

Completamos **audit/99-tp2.rules.base** como **audit/99-tp2.rules**, con exactamente dos reglas, y lo copiamos a **/etc/audit/rules.d/99-tp2.rules**. Las reglas se cargaron con **augenrules --load** y se listaron con **auditctl -l**. Los archivos están en **evidencias/parte3.1\_3.2\_Mauricio/**.

```
-w /srv/tp2/datos -p wa -k tp2_datos
-a always,exit -F arch=b64 -S execve -F path=/usr/local/bin/tp2-event -k tp2_exec
```

| Regla                                | Para qué está                                                                                                                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -w /srv/tp2/datos -p wa              | Vigila el directorio y todo lo que contiene. <b>w</b> registra escrituras y <b>a</b> , cambios de atributos (permisos, dueño). No se incluyen <b>r</b> ni <b>x</b> porque no se pedían y solo agregarían ruido.                   |
| -a always,exit -F arch=b64 -S execve | Regla de syscall: registra cada <b>execve</b> al salir de la llamada. <b>arch=b64</b> es necesario porque el número de cada syscall depende de la arquitectura (en x86_64, <b>execve</b> es la 59) y el entorno oficial es amd64. |
| -F path=/usr/local/bin/tp2-event     | Limita la regla a ese binario. Sin el filtro se registraría cada programa que se ejecute en el sistema.                                                                                                                           |
| -k tp2_datos / -k tp2_exec           | Claves para recuperar los eventos con <b>ausearch -k</b> .                                                                                                                                                                        |

| Evidencia             | Qué se intentó                                                                                                            | Qué se esperaba                                             | Qué ocurrió                                                                                                                                                                                                                                                                     | Qué demuestra                                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 01_reglas_cargadas    | Cargar las reglas y listarlas con <code>auditctl -l</code>                                                                | Que las dos reglas aparezcan activas en el kernel           | Aparecen las dos. El kernel devuelve la clave en forma normalizada: <code>-F key=tp2_exec</code>                                                                                                                                                                                | Las reglas no solo están escritas en el archivo: el kernel las está aplicando                                                  |
| 02_ausearch_tp2_datos | Agregar una línea a <code>publico.txt</code> con <code>tee -a</code> y cambiarle los permisos con <code>chmod 0640</code> | Un evento <code>tp2_datos</code> por cada tipo de operación | Dos eventos <code>syscall=257</code> ( <code>openat</code> de <code>tee</code> , flags <code>O_WRONLY O_CREAT O_APPEND</code> ) y uno <code>syscall=268</code> ( <code>fchmodat</code> de <code>chmod</code> , modo <code>0640</code> ), todos con <code>key="tp2_datos"</code> | La regla <code>-p wa</code> funciona en sus dos mitades: escritura ( <code>w</code> ) y cambio de atributos ( <code>a</code> ) |

Algunas aclaraciones sobre estas evidencias:

- **El primer `augenrules --load` falló.** Devolvió `Error sending add rule data request (No such file or directory)` en la línea 5 de `/etc/audit/audit.rules`, aunque las rutas vigiladas existían. El archivo se había editado con un editor de Windows (vía WinSCP) y tenía finales de línea CRLF: el `\r` quedaba pegado a la ruta y el kernel buscaba un archivo inexistente. Se corrigió con `dos2unix` y desde entonces el archivo se edita con `nano` dentro de la VM.
- **`uid=0` pero `auditd=1000`.** Las tres operaciones corrieron como root (con `sudo`), pero `auditd` conserva en `auditd` el usuario que inició la sesión (`alumno`). Esa trazabilidad es la que se pierde cuando se entra directamente como root por SSH (ver Parte 1).
- **La regla sobre el directorio también registra lo que hacen los programas.** Al ejecutar `tp2-event`, su escritura en `eventos.log`, que también está dentro de `/srv/tp2/datos`, generó un evento `tp2_datos` (se usa en la sección 4.2). Un watch sobre un directorio registra cualquier modificación dentro de él, no solo las que se hacen a mano.

## 4.2. Correlación entre `strace` y `auditd`

Ejecutamos `tp2-event` bajo `strace` con un mensaje identificable y después buscamos esa misma ejecución en `auditd`:

```
sudo strace -tt -o /srv/tp2/evidencias/03_strace_tp2event.txt \
    /usr/local/bin/tp2-event "correlacion_demo_1790287819"
sudo ausearch -k tp2_exec -ts recent
```

Respecto del comando de la consigna hicimos dos cambios. Agregamos **-tt**, porque sin esa opción strace no imprime la hora y el timestamp es uno de los datos a relacionar. Y no usamos el filtro **-e trace=**, así que el trace completo incluye también la llamada **getpid()**, que muestra el PID. Como **ausearch** se usó sin **-i**, los números de syscall y los argumentos se decodificaron a mano.

| Evidencia            | Qué se intentó                                            | Qué se esperaba                                          | Qué ocurrió                                                                                                                                                                                                                             | Qué demuestra                                                           |
|----------------------|-----------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 03_strace_tp2event   | Ejecutar <b>tp2-event</b> bajo strace                     | Ver el <b>execve</b> inicial y las syscalls del programa | La primera línea es <b>execve("/usr/local/bin/tp2-event", [..., "correlacion_demo_1790287819"]) = 0</b> a las 19:10:19.338120. Más abajo aparecen el <b>openat</b> sobre <b>eventos.log</b> , <b>getpid() = 23515</b> y el <b>write</b> | El detalle de la ejecución, visto en el momento desde fuera del proceso |
| 05_ausearch_tp2_exec | Buscar la misma ejecución con <b>ausearch -k tp2_exec</b> | Un evento <b>execve</b> del mismo proceso                | Registro SYSCALL con <b>syscall=59, success=yes exit=0, pid=23515 y exe="/usr/local/bin/tp2-event"</b> . El registro EXECVE muestra <b>a1="correlacion_demo_1790287819"</b>                                                             | La regla <b>tp2_exec</b> dejó un registro persistente de la ejecución   |
| 04_eventos_log       | Leer la última línea del log de la aplicación             | El PID que registró el propio programa                   | <b>2026-09-24T19:10:19-0300 pid=23515 uid=0 mensaje=correlacion_demo_1790287819</b>                                                                                                                                                     | Una tercera fuente, independiente de las otras dos, confirma el PID     |

La tabla resume los datos que pide relacionar la consigna:

| Dato       | strace (03)                             | auditd (05)                              | eventos.log (04) |
|------------|-----------------------------------------|------------------------------------------|------------------|
| Timestamp  | 19:10:19.338120                         | audit(1790287819.335:452) = 19:10:19.335 | 19:10:19         |
| PID        | getpid() = 23515                        | pid=23515                                | pid=23515        |
| Ejecutable | execve("/usr/local/bin/tp2-event", ...) | exe="/usr/local/bin/tp2-event"           | —                |
| Syscall    | execve                                  | syscall=59, arch=c000003e (x86_64)       | —                |

|           |                               |                                  |                                     |
|-----------|-------------------------------|----------------------------------|-------------------------------------|
| Argumento | "correlacion_demo_1790287819" | a1="correlacion_demo_1790287819" | mensaje=correlacion_demo_1790287819 |
| Resultado | = 0                           | success=yes exit=0               | línea escrita                       |

La misma ejecución se correlaciona también en una segunda syscall. strace muestra `openat(AT_FDCWD, "/srv/tp2/datos/eventos.log", O_WRONLY|O_CREAT|O_APPEND, 0666) = 3` a las 19:10:19.350918, y auditd registró con la clave `tp2_datos` el evento `audit(1790287819.347:453): syscall=257, a2=0x441, a3=0x1b6, exit=3, pid=23515 (06_ausearch_tp2_datos_tp2event)`. Los argumentos crudos de auditd decodifican exactamente a lo que strace muestra: `0x441` es `O_WRONLY|O_CREAT|O_APPEND` y `0x1b6` es `0666`.

#### 4.2.1 Análisis 5: correlación entre la syscall y el evento persistente

Tres fuentes independientes describen la misma ejecución: coinciden el PID (23515), el segundo, el ejecutable, el argumento y el resultado (`execve` devolvió 0). Además, los argumentos crudos de auditd coinciden exactamente con los que strace muestra decodificados, así que no son dos ejecuciones parecidas sino la misma.

La diferencia está en cómo se obtiene cada registro. strace observa activamente un proceso: lo lanza, o se adjunta a él, y el kernel lo detiene en cada syscall para avisarle mediante `ptrace`. Por eso en auditd el padre del proceso (`ppid=23512`) es el propio strace. Muestra todas las syscalls con sus argumentos, pero lo que registra es efímero: si nadie estaba trazando, esa información no existe.

auditd, en cambio, aplica reglas cargadas de antemano en el kernel. Registra solo lo que esas reglas seleccionan, pero lo hace aunque nadie esté mirando. El registro persiste en `/var/log/audit/audit.log` e incluye contexto de identidad que strace no muestra, como el `auid`.

En los timestamps se ve una diferencia sistemática: en las dos syscalls correlacionadas, auditd marca entre 3 y 4 ms antes que strace (`execve`: .335 contra .338; `openat`: .347 contra .351). auditd toma la hora dentro del kernel al procesar la syscall; strace la toma en el proceso trazador, después de ser notificado. Que el desfase sea constante refuerza que se trata de los mismos eventos.

En síntesis, strace sirve para entender qué hace un programa, y auditd, para poder demostrar después qué ocurrió.

### 4.3. Línea BASE y control de integridad con AIDE

#### 4.3.1 Objetivo

El objetivo de esta etapa fue implementar un mecanismo de control de integridad sobre los archivos ubicados en `/srv/tp2/datos`. Para ello se configuró AIDE con una base de referencia que permitiera guardar el estado del directorio y posteriormente detectar modificaciones dentro del mismo, ya sea creación de nuevos archivos, cambios en atributos y contenido de los archivos.

### 4.3.2 Configuración de AIDE

Como primer paso, en base al archivo **aide/aide.conf.base**, se creó el archivo **aide/aide.conf** como configuración local en la cual se estableció cuál será la base activa, la base nueva y por donde se mostrará el reporte.

```
database_in=file:/srv/tp2/aide/aide.db
database_out=file:/srv/tp2/aide/aide.db.new
report_level=changed_attributes
report_url=stdout

# definición de regla TP2NORMAL para verificar permisos, tipo, inode, enlaces,
# propietario, grupo, tamaño, tiempos y SHA-256.
TP2NORMAL = p+ftype+i+l+u+g+s+m+c+acl+sha256

# aplicación de regla TP2NORMAL solamente a /srv/tp2/datos.
/srv/tp2/datos TP2NORMAL
```

La configuración principal es la regla **TP2NORMAL** la cual fue aplicada exclusivamente sobre **/srv/tp2/datos**, evitando analizar innecesariamente el resto del sistema.

| Cambios que permite detectar cada parámetro |                                    |
|---------------------------------------------|------------------------------------|
| p                                           | En los permisos                    |
| ftype                                       | En el tipo de archivo              |
| i                                           | Inodo                              |
| l                                           | Enlaces                            |
| u                                           | De propietario                     |
| g                                           | De grupo                           |
| s                                           | En el tamaño del archivo           |
| m                                           | Fecha de modificación              |
| c                                           | Fecha de modificación de metadatos |
| acl                                         | Cambios en ACL                     |
| sha256                                      | En el contenido                    |

### 4.3.3 Generación de la línea base

Una vez instalada y validada, se inicializó la configuración con la nueva base de datos de AIDE ( evidencias/parte3.3\_Gaston/aide\_init.txt). Esta base representa el estado considerado correcto de los archivos monitoreados y será utilizada como referencia para detectar modificaciones posteriores.

#### 4.3.4 Cambios controlados

Luego de la configuración inicial se solicitó realizar tres cambios dentro del directorio al cual se le aplicó la regla para evidenciar la detección por parte de AIDE de la modificación.

| Cambio Realizado                           | Evidencia                                                                                                                                                 |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Creación de un archivo                     | Added entries:<br>f+++++: /srv/tp2/datos/nuevo_archivo.txt                                                                                                |
| Modificación en el contenido de un archivo | File: /srv/tp2/datos/publico.txt<br>Size : 58   90<br>SHA256: E2KNvjylpgtXS1KwjRl05dCyF6E8jzHHt7vekpZGjs=  <br>FJGtJM6GTUCZvEBnpH9rHlqvXd8fwtgpMiKSwgEfU= |
| Cambiar los permisos de un archivo         | File: /srv/tp2/datos/confidencial.txt<br>Perm: -rw-r--r--   -rw-r----                                                                                     |

Dentro de la evidencia se puede ver que AIDE detectó la creación de un nuevo archivo el cual no existía cuando se creó la línea base y los cambios realizados en otros dos archivos.

En cuanto a la modificación de permisos AIDE muestra que no solo verifica cambios en el contenido de un archivo sino que también lo hace sobre sus atributos y es capaz de detectar el cambio de tamaño de un archivo como ocurrió con `publico.txt` el cual cambió su tamaño de 58 a 90 bytes y cambio si hash SHA-265.

Los resultados obtenidos coinciden con los cambios realizados intencionalmente. AIDE detectó tanto la incorporación de un nuevo archivo como modificaciones en atributos y contenido. Esto demuestra que la configuración utilizada permite controlar la integridad de los archivos mediante la comparación con la línea base.

#### 4.3.5 Limitación de AIDE

AIDE permitió detectar tres tipos de modificaciones: la creación de un archivo nuevo, el cambio de permisos de un archivo existente y la modificación del contenido de otro archivo. Sin embargo, una limitación del esquema utilizado es que la base de referencia se almacena en la misma máquina que se está protegiendo. Si un atacante obtuviera privilegios suficientes, podría modificar o eliminar tanto los archivos monitoreados como la propia base de AIDE. En ese escenario, la herramienta perdería parte de su capacidad para garantizar la integridad de la referencia. Por este motivo, en un entorno real sería

conveniente proteger la base de referencia mediante almacenamiento externo o mecanismos que impidan su modificación por el sistema comprometido.

## 5. Escenario integrador

Para el escenario integrador usamos una sola VM con los tres controles activos a la vez: el perfil de AppArmor en enforce, las dos reglas de auditd cargadas y una línea base de AIDE recién creada. La preparación, la ejecución de `scripts/escenario_final.sh` y la recolección de evidencia se hicieron con un script propio (`correr_escenario_integrador.sh`), que deja todas las salidas en `evidencias/escenario_integrador_20261009_000325/`. La ejecución fue el 09/10 a las 00:03, en la VM `alumno-virtualbox` (Ubuntu 20.04.2 LTS, kernel 5.4.0-70, AIDE 0.16.1, según `00_entorno`).

| Control  | Estado antes del escenario                                                                                           | Evidencia                                                              |
|----------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| AppArmor | <code>apparmor_parser -Q</code> sin errores; el perfil de <code>tp2-reader</code> figura como <code>(enforce)</code> | 01_apparmor_enforce                                                    |
| auditd   | <code>auditctl -l</code> muestra las dos reglas: <code>tp2_datos</code> y <code>tp2_exec</code>                      | 02_auditd_reglas                                                       |
| AIDE     | Base creada con 4 entradas; el chequeo inmediato dio «AIDE found NO differences»                                     | 03a_aide_config_check,<br>03b_aide_init,<br>03c_aide_check_base_limpia |

La tabla reconstruye lo que ocurrió en cada paso del escenario. Todos los eventos están en el mismo segundo (00:03:29) y AIDE se ejecutó tres segundos después:

| Paso del escenario                          | Qué se vio en la terminal                                    | Qué quedó registrado                                                                                                                                                                   | Control que actuó  |
|---------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 1. Lectura de <code>publico.txt</code>      | Se imprimió el contenido                                     | Ningún evento de AppArmor: el perfil permite esa lectura                                                                                                                               | AppArmor (permite) |
| 2. Lectura de <code>confidencial.txt</code> | <code>Permission denied</code>                               | <code>apparmor="DENIED" operation="open" name="/srv/tp2/datos/confidencial.txt", pid 40788, fsuid=0, 00:03:29.206 (05_apparmor_avc)</code>                                             | AppArmor (bloquea) |
| 3. Ejecución de <code>tp2-event</code>      | Evento registrado en <code>/srv/tp2/datos/eventos.log</code> | <code>tp2_exec: execve del pid 40789, success=yes, a1=escenario_integrador_20261009_000329 (06). tp2_datos: openat de eventos.log del mismo pid, O_WRONLY O_CREAT O_APPEND (07)</code> | auditd             |

|                          |            |                                                                                                                                                                          |        |
|--------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 4. Cambio en publico.txt | Sin salida | tp2_datos: openat de publico.txt hecho por bash (pid 40785, la redirección >> del propio script) y fchmodat con modo 0640 hecho por chmod (pid 40790), 00:03:29.218 (07) | auditd |
| Chequeo posterior        | —          | Changed entries: 2. publico.txt: tamaño 159 → 193, Mtime, Ctime y SHA256. eventos.log: tamaño 335 → 421, Mtime, Ctime y SHA256 (08_aide_check)                           | AIDE   |

Algunas aclaraciones sobre estas evidencias:

- **El chmod quedó en auditd, pero AIDE no informa un cambio de permisos.** publico.txt ya tenía modo 0640 desde las pruebas de la sección 4.1: el Ctime de la base (2026-09-24 19:47:54) es el chmod de la evidencia 02. El escenario volvió a aplicar el mismo modo. auditd registra la operación (fchmodat exitoso), pero AIDE compara estados y el permiso final es igual al de la base, así que solo informa el cambio de Ctime. auditd registra acciones; AIDE, diferencias de estado.
- **AIDE también marcó eventos.log.** tp2-event le agregó una línea, que es un cambio legítimo, pero con la regla TP2NORMAL se informa igual que una alteración. Para un archivo de log correspondería una regla distinta, por ejemplo una que solo verifique que el archivo crezca.
- **Root no evita el bloqueo.** El escenario se ejecuta con sudo y el evento DENIED tiene fsuid=0, igual que en la evidencia 03 de la Parte 2.
- **La recarga de las reglas también queda auditada.** Al cargarlas, auditd registró eventos CONFIG\_CHANGE (remove\_rule y add\_rule) con audit=alumno. Si alguien desactivara las reglas para ocultar una acción, ese cambio dejaría rastro.
- **AIDE 0.16.1 no acepta la configuración del repo.** Rechazó la línea 7 (database\_in): esa opción, report\_level y ftype son de versiones más nuevas, como la 0.18.6 de la VM de la Parte 3.3. Se usó una copia compatible con database en lugar de database\_in, sin report\_level y sin ftype (NOTA\_aide\_compatibilidad). Los tiempos m+c sí se aplicaron, y por eso el reporte muestra Mtime y Ctime.
- **La primera ejecución falló.** escenario\_final.sh se cortó en la línea 2 con set: pipefail: nombre de opción no válido porque la copia del repo en la VM tenía finales de línea CRLF. Es el mismo problema de la sección 4.1. Se convirtieron los archivos a LF y se repitió la ejecución completa.

## 6. Conclusión

El escenario integrador permite reconstruir lo ocurrido combinando las tres fuentes. AppArmor permitió la lectura de `publico.txt` y bloqueó la de `confidencial.txt`, aunque el proceso corría como root. auditd registró la ejecución de `tp2-event` y las tres modificaciones dentro de `/srv/tp2/datos`, con el proceso, el usuario que inició la sesión y el resultado de cada una. Tres segundos después, AIDE detectó que `publico.txt` y `eventos.log` ya no coincidían con la línea base.

Cada control responde una pregunta distinta, y ninguno alcanza solo. El hardening reduce la superficie de ataque antes de que ocurra algo. AppArmor impide que una aplicación exceda lo que su perfil autoriza, pero solo deja rastro de lo que bloquea. auditd dice qué syscall se ejecutó, quién la hizo, cuándo y con qué resultado, pero solo de lo que sus reglas seleccionan. AIDE detecta que el estado del sistema cambió, aunque no sabe cómo, y no registra una acción que deja el estado igual, como el `chmod` repetido del escenario. strace sirve para entender qué hace un programa, pero no deja evidencia por sí mismo.

El trabajo también mostró que obtener evidencia confiable exige verificar el entorno: la VM no tenía la versión de Ubuntu ni de AIDE que pedía la consigna, los eventos de AppArmor no aparecían donde la consigna indicaba buscarlos y los finales de línea CRLF rompieron dos veces la carga de archivos. Documentar esas diferencias fue parte del resultado.